

XML

DATEN

- Unstrukturiert (z.B. )
- Semi-Strukturiert (z.B. )
- Strukturiert (z.B. )



DHBW
Duale Hochschule
Baden-Württemberg

XML

... ist eine Auszeichnungssprache sowie ein Datenaustauschformat und ist

- Maschinenlesbar und interpretierbar
- Semi-strukturiert



PIZZA ONLINE BESTELLEN

Welche Daten fallen an?

UNSTRUKTURIERT

Hi, ich hätte gerne *eine extragroße Pizza Funghi mit zusätzlich Artischocken*. Bitte liefern zu *Lieschen Müller* in die *Erzbergerstraße 1337*. Pronto!

SEMI-STRUKTURIERT

```
1 <pizza>
2   <art>funghi</art>
3   <groesse>extragroß</groesse>
4   <extra_zutaten>
5     <zutat>artischocken</zutat>
6   </extra_zutaten>
7   <lieferort>
8     <name>Lieschen Müller</name>
9     <strasse>Erzbergerstraße
10    1337</strasse>
11   </lieferort>
12   <lieferzeit>sofort</lieferzeit>
13 </pizza>
```

Frage: Warum ist XML "*nur*" semi-strukturiert?

XML - ÜBERSICHT

- extensible Markup Language
- Auszeichnungssprache
 - d.h. sie ist nicht ausführbar und tut nichts
- "Extensible", also erweiterbar
 - es können eigene Elemente hinzugefügt werden
 - daher nur semi-strukturiert
- Entworfen als Web Standard in 1996 vom World Wide Web Consortium (W3C)

DESIGN ZIELE

1. XML shall be straightforwardly usable over the Internet.
2. XML shall support a wide variety of applications.
3. XML shall be compatible with SGML.
4. It shall be easy to write programs which process XML documents.
5. The number of optional features in XML is to be kept to the absolute minimum, ideally zero.
6. XML documents should be human-legible and reasonably clear.
7. The XML design should be prepared quickly.
8. The design of XML shall be formal and concise.
9. XML documents shall be easy to create.
10. Terseness in XML markup is of minimal importance.

Quelle: W3C Empfehlung

LITERATUR

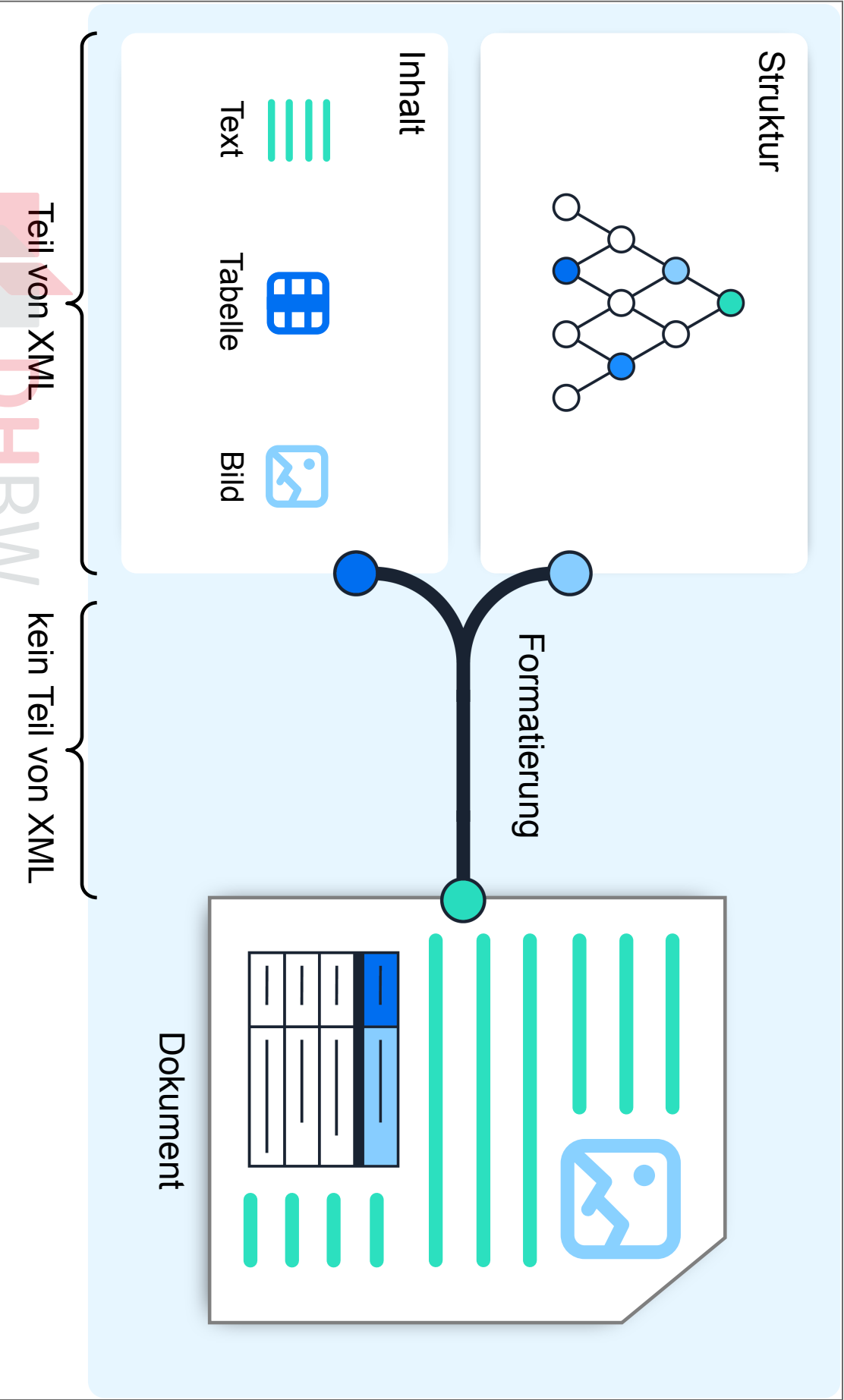
- Webseiten des W3-Konsortiums (w3.org)
- Tutorials:
 - W3Schools: Introduction to XML
 - MDN: XML introduction
- Bücher
 - Verhoeogen, Helmut: Einstieg in XML. Bonn: Galileo Press GmbH.
 - Sebestyen, Thomas J.: XML: Einstieg für Anspruchsvolle, Pearson Studium.

TOOLS

- Editoren
 - Beliebige Code-Editoren: VS Code, Notepad++, ...
 - Spezialisierte Editoren: Altova XMLSpy, XMLNotepad, ...
 - Online-Editoren: JSONFormatter: XML Editor, xmleditoronline.org, ...
- Validierung
 - W3C Validator
 - Truugo XML, DTD, XSD Validator

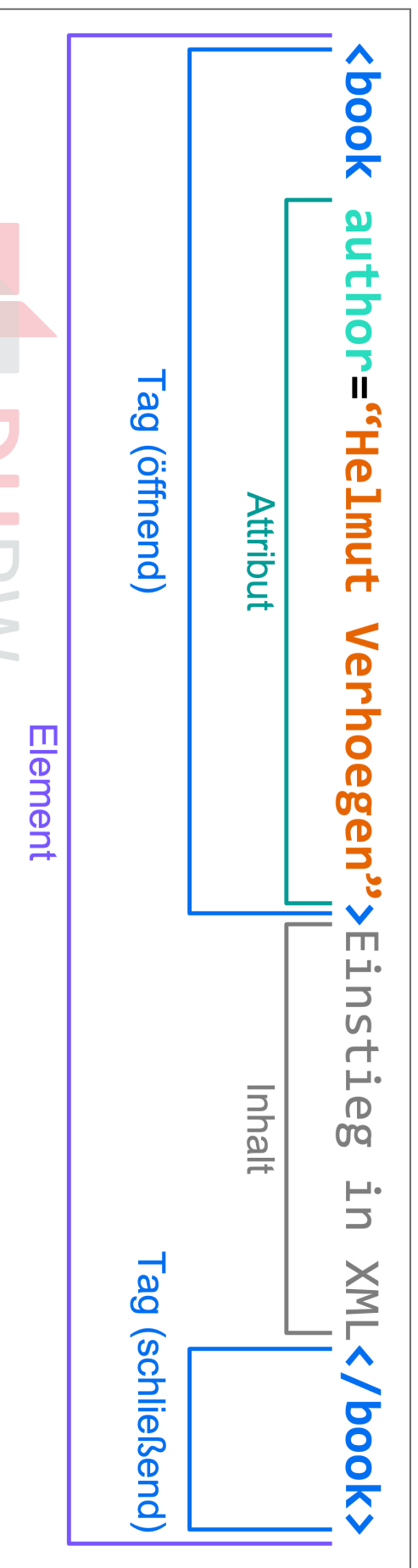
XML AUFBAU UND SYNTAX

WAS IST EIN DOKUMENT?



XML ELEMENTE

- Ein XML-Dokument besteht aus *Elementen*
- Elemente sind definiert durch *Tags*
- Elemente haben i.d.R. einen Inhalt
- Elemente können leer sein
- Elemente können Attribute besitzen, welche Eigenschaften des Elements beschreiben

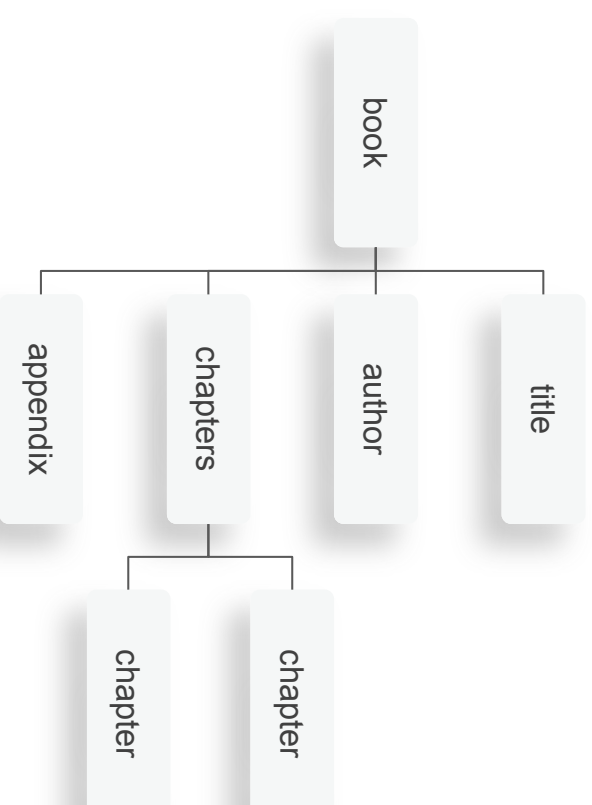


XML HIERARCHIE

XML-DOKUMENT

```
1<book>
2  <author>...</author>
3  <title>...</title>
4  <chapters>
5    <chapter>
6      <title>...</title>
7      <content>...</content>
8    </chapter>
9    <chapter>
10     <title>...</title>
11     <content>...</content>
12   </chapters>
13  <appendix>...</appendix>
14</book>
```

HIERARCHIE



STRUKTUR EINES XML-DOKUMENTS

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!-- XML-Deklaration -->
3
4  <?xmlstylesheet type="text/css" href="mystyle.css"?>
5  <!-- Verarbeitungsanweisung (optional) -->
6
7  <!DOCTYPE book SYSTEM "book.dtd">
8  <!-- Dokumenttypdeklaration mit optionalem Verweis auf DTD -->
9
10 <!-- XML-Daten (getagelter Text) -->
11 <book title="Einführung in XML" author="Max Mustermann">
12
13 </book>
```



DHBW

Duale Hochschule
Baden-Württemberg

ÜBUNG

Was ist das Problem mit folgendem XML-Dokument?



```
1 <Book>
2   <Title>Mathe für Ingenieure 1 & 2 Lernhefte Set</Title>
3   <Author>Daniel Jung</Author>
4   <ISBN>9783947506378</ISBN>
5   <Inhalt>
6     <Kapitel>Mathematischer Werkzeugkoffer</Kapitel>
7     <Kapitel>Warum ist 0 < 1?</Kapitel>
8   </Inhalt>
9 </Book>
```

Parse XML

ÜBUNG

Wir ersetzen das &-Symbol



```
1 <Book>
2   <Title>Mathe für Ingenieure 1 und 2 Lernhefte Set</Title>
3   <Author>Daniel Jung</Author>
4   <ISBN>9783947506378</ISBN>
5   <Inhalt>
6     <Kapitel>Mathematischer Werkzeugkoffer</Kapitel>
7     <Kapitel>Warum ist 0 < 1?</Kapitel>
8   </Inhalt>
9 </Book>
```

Parse XML



DHBW

Duale Hochschule
Baden-Württemberg

ÜBUNG - WORKAROUND

Wir ersetzen das unvollständige Tag beginnend mit <

```
1  <Book>
2    <Title>Mathe für Ingenieure 1 und 2 Lernhefte Set</Title>
3    <Author>Daniel Jung</Author>
4    <ISBN>9783947506378</ISBN>
5    <Inhalt>
6      <Kapitel>Mathematischer Werkzeugkoffer</Kapitel>
7      <Kapitel>Warum ist 0 kleiner als 1?</Kapitel>
8    </Inhalt>
9  </Book>
```

Parse XML

Was ist, wenn wir die Zeichen nicht umschreiben können oder wollen?

XML-INHALT & ESCAPING

- Inhalt sind (fast) alle beliebigen Zeichen aus dem Unicode-Alphabet
- Einige Zeichen sind reserviert und müssen escaped werden:
 - `<` → `<`;
 - `>` → `>`;
 - `&` → `&`;
 - `"` → `"`;
 - `'` → `'`;

ÜBUNG - LÖSUNG

```
1  <Book>
2    <Title>Mathe für Ingenieure 1 &amp; 2 Lernhefte Set</Title>
3    <Author>Daniel Jung</Author>
4    <ISBN>9783947506378</ISBN>
5    <Inhalt>
6      <Kapitel>Mathematischer Werkzeugkoffer</Kapitel>
7      <Kapitel>Warum ist 1 &gt; 0?</Kapitel>
8    </Inhalt>
9  </Book>
10
```

Parse XML

XML-KOMMENTARE

- Dürfen überall stehen, wo Text oder Elemente stehen dürfen
- Werden vom Parser in der Verarbeitung ignoriert
- Beginnen mit `<!--` und enden mit `-->`
- Sind auch über mehrere Zeilen möglich

```
1 <book>  
2 <!-- Das ist ein Kommentar innerhalb eines book-Elements -->  
3 </book>  
4 <!--  
5     Das ist ein Kommentar  
6     über mehrere Zeilen  
7 -->
```

XML-WHITESPACE

- Whitespace = Leerzeichen, Tabs, Zeilenumbrüche
- Für die Verarbeitung gilt:
 - Außerhalb von Elementen darf beliebig Whitespace verwendet werden
 - 🖐️ wird für Verarbeitung ignoriert
 - Innerhalb von Elementen zählt Whitespace als Inhalt und wird beibehalten
- Whitespace kann und soll verwendet werden, um die Lesbarkeit zu erhöhen

CDATA-ABSCHNITTE

- CDATA steht für "Character Data"
- Erlaubt das Einfügen von Text, der nicht geparkt wird
- Beginnen mit <![CDATA[und enden mit]]>
- Wird verwendet, um Zeichen zu verwenden, die sonst escaped werden müssten
- Innerhalb ist alles erlaubt außer]]>

```
1 <book>
2   <title>Einführung in XML</title>
3   <description>
4     <![CDATA[
5       Dies ist ein Text, der nicht geparkt wird.
6       Reservierte Zeichen wie >, <, &, " und
7       müssen nicht escaped werden.
8     ]]>
9   </description>
10 </book>
```

XML-REGELN

- ! Ein XML-Dokument muss *genau ein* Wurzelement haben
- ! XML ist case-sensitive, d.h. Groß- und Kleinschreibung unterscheiden sich
- ! Elemente dürfen nicht überlappen
- ! Tags müssen immer geschlossen werden
- ! Elemente müssen korrekt geschachtelt sein, d.h. sie müssen in der richtigen Reihenfolge geschlossen werden
- ! Attribute müssen in (doppelten) Anführungszeichen stehen

XML IM ALLTAG

- XML wird in vielen Bereichen eingesetzt, z.B.:
 - Webentwicklung (z.B. XHTML, SVG)
 - Datenbanken (z.B. XML-Datenbanken)
 - Konfigurationsdateien (z.B. Maven, Android)
 - Dokumentenaustausch (z.B. Office Open XML, OpenDocument)
- XML ist ein Standardformat für den Datenaustausch zwischen Systemen:
 - SOAP-API
 - RSS/Atom-Feeds

BEISPIELE: XML IM ALLTAG

WEBENTWICKLUNG

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport"
6       content="width=device-width, initial-
7       scale=1.0">
8     <title>Hello World! Site
9     Title</title>
10   </head>
11   <body>
12     <h1>Hello World!</h1>
13   </body>
14 </html>
```

KONFIGURATIONSDATEI

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project
3   xmlns="http://maven.apache.org/POM/4.
4   0.0"
5   xmlns:xsi="http://www.w3.org/2001/XML
6   Schema-instance
7   xsi:schemaLocation="http://maven.apac
8   he.org/POM/4.0.0
9   http://maven.apache.org/maven-
10  v4_0_0.xsd">
11   <modelVersion>4.0.0</modelVersion>
12   <groupId>com.sample.project</groupId>
13   <artifactId>core</artifactId>
14   <version>1.0.0-SNAPSHOT</version>
15   <name>Component Name</name>
16   <properties>
17     <java.version>17</java.version>
18   </properties>
19   <project.build.sourceEncoding>UTF-
20   8</project.build.sourceEncoding>
21   <dependencies>
22     <dependency>
23       <groupId>junit</groupId>
```

```

20 <artifactId>junit</artifactId>
21 <version>4.12</version>
22 <scope>test</scope>
23 </dependency>
24 <dependency>
25 <groupId>org.assertj</groupId>
26 <artifactId>assertj-
27 core</artifactId>
28 <version>3.6.2</version>
29 <scope>test</scope>
30 </dependency>
31 </dependencies>
32
33 <build>
34 <plugins>
35 <plugin>
36 <artifactId>maven-
37 compiler-plugin</artifactId>
38 <version>3.7.0</version>
39 <configuration>
40 <source>${java.version}</source>
41 <target>${java.version}</target>
42 </plugin>
43 </plugins>
44 </build>

```



XML ≠ XML

order_type_1.xml

```
1  <pizza>
2    <art>funghi</art>
3    <groesse>extra</groesse>
4    <extra_zutaten>
5      <zutat>artischocken</zutat>
6    </extra_zutaten>
7    <liefert>
8      <name>Lieschen Müller</name>
9      <strasse>Erzbergerstraße
10     </strasse>
11     <liefert>
12   </pizza>
```

order_type_2.xml

```
1  <pizza art="funghi" groesse="xxl">
2    <belage>
3      <belag>artischocken</belag>
4    </belage>
5    <anschrift>
6      <vorname>Lieschen</vorname>
7      <nachname>Müller</nachname>
8    </anschrift>
9    <strasse>Erzbergerstraße</strasse>
10   <nr>1337</nr>
11   <lieferrn_in>-1</lieferrn_in>
12 </pizza>
```

Was sind die Unterschiede in den folgenden Pizza-Bestellungen?

DATA TYPE DEFINITION

- Data Type Definition = DTD
 - Beschreibung der Dokumentstruktur für eine Klasse von Dokumenten
 - Die DTD legt fest
 - welche Elemente erlaubt sind 🙋 ELEMENT
 - welche Inhalte Elemente haben dürfen
 - wie Elemente verschachtelt sein dürfen (welche Beziehungen sie zueinander haben)
 - welche Attribute es geben darf 🙋 ATTLIST
 - welche Werte ein Attribut annehmen darf
 - welche (komplexeren) Bausteine es gibt 🙋 ENTITY
- Empfohlene Literatur: [W3Schools: DTD Tutorial](#)

DTD-BEISPIEL

XML-DOKUMENT

```
1 <pizza art="funghi"  
2   groesse="xxl">  
3   <belaege>  
4     <belag>artischocken</belag>  
5   </belaege>  
6   <anschrift>  
7     <vorname>Lieschen</vorname>  
8     <nachname>Müller</nachname>  
9   </anschrift>  
10  <strasse>Erzbergerstraße</strasse>  
11  <nr>1337</nr>  
12 </pizza>
```

DTD-DATEI

```
1 <!DOCTYPE pizza [  
2   <!ELEMENT pizza (belaege, anschrift,  
3     liefern_in)*>  
4   <!ATTLIST pizza  
5     art CDATA #REQUIRED  
6     groesse CDATA #REQUIRED  
7   >  
8   <!ELEMENT belaege (belag)*>  
9   <!ELEMENT belag (#PCDATA)>  
10  <!ELEMENT anschrift (vorname, nachname,  
11    strasse, nr)>  
12  <!ELEMENT vorname (#PCDATA)>  
13  <!ELEMENT nachname (#PCDATA)>  
14  <!ELEMENT strasse (#PCDATA)>  
15  <!ELEMENT nr (#PCDATA)>  
16  <!ELEMENT liefern_in (#PCDATA)>  
17 ]>
```

DTD - SYNTAX

ELEMENTE

- Der Elementname
 - ⚠ beginnt mit einem Buchstaben oder Unterstrich
 - ⚠ kann danach zusätzlich Zahlen, Bindestriche und Punkte enthalten
 - ⚠ ist case-sensitive
 - ⚠ darf keine Leerzeichen enthalten
 - ⚠ darf nicht mit der Buchstabenkombination XML beginnen
 - !⚠ Doppelpunkte sollten nur mit Namespaces (Namensräumen) verwendet werden

ÜBUNG

Welche Elementnamen sind gültig?

<Name1> ✓

<Complete Name> ✗

<Name.1> ✓

<_Name> ✓

<Name=Max> ✗

<Name-1> ✓

< Name > ✗

<1Name> ✗

<XMLName> ✗

<CompleteName> ✓

DTD - SYNTAX

ELEMENTE

Syntax: `<!ELEMENT Elementname (Inhalt)*>`

Leeres Element	<code><!ELEMENT Elementname EMPTY></code>
Element mit Textinhalt	<code><!ELEMENT Elementname (#PCDATA)></code>
Element mit jeglichem Inhalt	<code><!ELEMENT Elementname ANY></code>
Element mit Kindelementen	<code><!ELEMENT Elementname (Kind1, Kind2, Kind3)></code>
Element mit gemischtem Inhalt	<code><!ELEMENT Elementname (Kind1, Kind2, #PCDATA)></code>

DTD - SYNTAX

KARDINALITÄTEN

1..1 ➡ Genau einmal	<code><!ELEMENT person (name, age, height)></code>
0..1 ➡ Optional	<code><!ELEMENT person (name, age, partner?)></code>
1..* ➡ Mindestens einmal	<code><!ELEMENT book (titel, chapter+)></code>
0..* ➡ Optional, aber beliebig oft	<code><!ELEMENT pizza (topping, extraTopping*)></code>

DTD - SYNTAX

ATTRIBUTLISTE

Syntax: <!ATTLIST Elementname Name Typ Wert>

ATTRIBUTTYPEN

CDATA	Beliebiger Textinhalt
ID	Eindeutige ID innerhalb des Dokuments
IDREF	Referenz auf ein anderes Element mit ID
IDREFS	Eine Liste von IDREF
ENTITY	Name einer vordefinierten, externen Entität
ENTITIES	Eine Liste von ENTITY
ENUMERATION	Eine Liste von erlaubten Werten

ATTRIBUTWERTE

<Wert>	Standardwert des Attributs
#REQUIRED	Attribut muss angegeben werden
#IMPLIED	Attribut ist optional
#FIXED <wert>	Attribut hat einen festen Wert, der nicht geändert werden kann

DTD - SYNTAX

ATTRIBUTLISTE - BEISPIEL



```
1 <!ATTLIST recepie
2   id      ID          #REQUIRED
3   cookbook IDREF      #REQUIRED
4   author  CDATA       #IMPLIED
5   type    (cooking | baking) "cooking"
6   title   CDATA       #IMPLIED>
```

PCDATA VS. CDATA

PCDATA $\neq$ CDATA

- PCDATA
 - 👉 steht für "Parsed Character Data"
 - 👉 Text, der vom Parser interpretiert (geparst) wird
 - 👉 Reservierte Zeichen müssen escaped werden
- CDATA
 - 👉 steht für "Character Data"
 - 👉 Text, der nicht geparst wird
 - 👉 Reservierte Zeichen müssen nicht escaped werden

ELEMENTE VS. ATTRIBUTE

Es gibt keine Regel 🙅 Personal Preference

- 💡 Attribute für Metadaten (also Infos über das Element)
 - 💡 Elemente für Inhalt (also eigentliche Daten des Dokuments)
 - 💡 Elemente für Listen/Aufzählungen
 - 💡 Elemente für hierarchische Strukturen
- Generelle Empfehlung:**

XML UND DTD - BEISPIEL

DTD-DEFINITION



```
1 <!ELEMENT lunch (drink*, meal+)>
2 <!ELEMENT drink (#PCDATA)>
3 <!ATTLIST drink size
4 (small|medium|large) #REQUIRED>
5 <!ELEMENT meal (#PCDATA)>
  <!ATTLIST meal isVegan (yes|no)
    "no">
```

XML-DOKUMENT

```
1 <?xml version="1.0" encoding="UTF-
2 8"?>
3 <!DOCTYPE lunch SYSTEM "lunch.dtd">
4 <lunch>
5   <drink size="medium">Cola</drink>
6   <drink size="large">Wasser</drink>
7   <meal isVegan="yes">French
8   Fries</meal>
9   <meal>Chicken Nuggets</meal>
10 </lunch>
```

DTD-EINBINDUNG

INTERN

```
1 <!DOCTYPE book [
2   ...
3 ]>
4 <!-- restliches Dokument -->
```

oder EXTERN

```
1 <!DOCTYPE book SYSTEM
2   "path_to_book_dtd.dtd">
3 <!-- restliches Dokument -->
```


NACHTEILE VON DTD

- ⚠ Wiederverwendung bzw. Erweiterung ist schwierig
 - ➡ Keine Möglichkeit, nur einen Teil einer DTD zu verwenden
 - ➡ Keine Vererbung
- ⚠ Keine Datentypen/Wertebereiche
- ⚠ Keine komplexeren Kardinalitäten

Alternative: XML Schema Definition (XSD). bietet mehr Möglichkeiten und Flexibilität

WOHLGEFORMTES VS. VALIDES XML

WELL FORMED

Ein Dokument ist *wohlgeformt*, wenn...

- ...es der XML-Syntax genügt
- ...es genau ein Wurzelelement hat
- ...es korrekte Schachtelung aufweist

VALID

Ein Dokument ist *valide*, wenn...

- ...es wohlgeformt ist
- ...es auf eine existierende DTD verweist
- ...dessen Inhalt der DTD entspricht

ÜBUNG: WOHLGEFORMT?

- ➡ `<user phone=049176129537></user>` ❌
- ➡ `<user e-mail='sara.softwareengineer@gmail.com'></user>` ✔️
- ➡ `<user e-mail="sara.softwareengineer@gmail.com"> </user>` ✔️
- ➡ `<user e-mail="garkeineemail" semester='second'></user>` ✔️
- ➡ `<dhbw location="Karlsruhe" location="Deutschland"></dhbw>` ❌
- ➡ `<Car type="compact" brand="Vw">polo</car>` ❌
- ➡ `<vehicle is-electric="true" brand="tesla">3</vehicle>` ✔️
- ➡ `<Book xml-related="yes" title="XML Grundlagen"></Book>` ❌
- ➡ `<Pizza Menu>🍕</Pizza Menu>` ❌

XPATH

Eine Query Language (Abfragesprache) zum Durchsuchen von XML Dokumenten

BEISPIEL

```
1  <!-- all Extrazutaten der Bestellung -->
2  <!-->
3  /order/pizza//ingredient/text()
4
5  <!-- alle extragroßen Pizzen der
6  Bestellung -->
7  /order/pizza[@size="xxl"]
8
9  <!-- Anzahl der Pizzen in der Bestellung
10 -->
11 count(//pizza)
12
13
```

💡 Weitere Infos hier.

```
1  <!-- all Extrazutaten der Bestellung -->
2  <!-->
3  /order/pizza//ingredient/text()
4
5  <!-- alle extragroßen Pizzen der
6  Bestellung -->
7  /order/pizza[@size="xxl"]
8
9  <!-- Anzahl der Pizzen in der Bestellung
10 -->
11 count(//pizza)
```

 [XPath Tester WebToolkitOnline](#)

 [XPath Tester FreeFormatter](#)

XPATH - SYNTAX

SELEKTOREN

```
<?xml version="1.0"
encoding="UTF-8"?>
<bookstore>
  <book>
    <title lang="en">Harry
Potter</title>
    <price>29.99</price>
  </book>
  <book>
    <title
lang="en">Learning
XML</title>
    <price>39.95</price>
  </book>
</bookstore>
```

Selektor	Beispiel	Erklärung
<i>Elementtyp</i>	book	Wählt alle <book> Elemente
/	/<bookstore>/<book>	Auswahl auf nächster Hierarchieeben
//	//title	Auswahl auf beliebiger Hierarchieeben
Auswahl		
aktueller		
Knoten		
/bookstore/book/.		

..	/bookstore/book/..	Auswahl Elternknoten
@attributname	//title/@lang	Attributauswahl
text()	//title/text()	Textauswahl

XPATH - SYNTAX

PRÄDIKATE



DHBW
Duale Hochschule
Baden-Württemberg



```

<?xml version="1.0"
encoding="UTF-8"?>
<bookstore>
  <book>
    <title lang="en">Harry
Potter</title>
    <price>29.99</price>
  </book>
  <book>
    <title
lang="en">Learning
XML</title>
    <price>39.95</price>
  </book>
</bookstore>

```

Prädikattyp	Beispiel	Erklär
Index	//book[1]	SE EI
Attribut	//title[@lang='en']	SE EI AI
Kindelement	//book[price > 30]	SE EI Ü in
Funktion	//book[last()] //book[contains(title/@lang,"e")]	SE EI EI FI

XPATH - SYNTAX

RELATIONEN

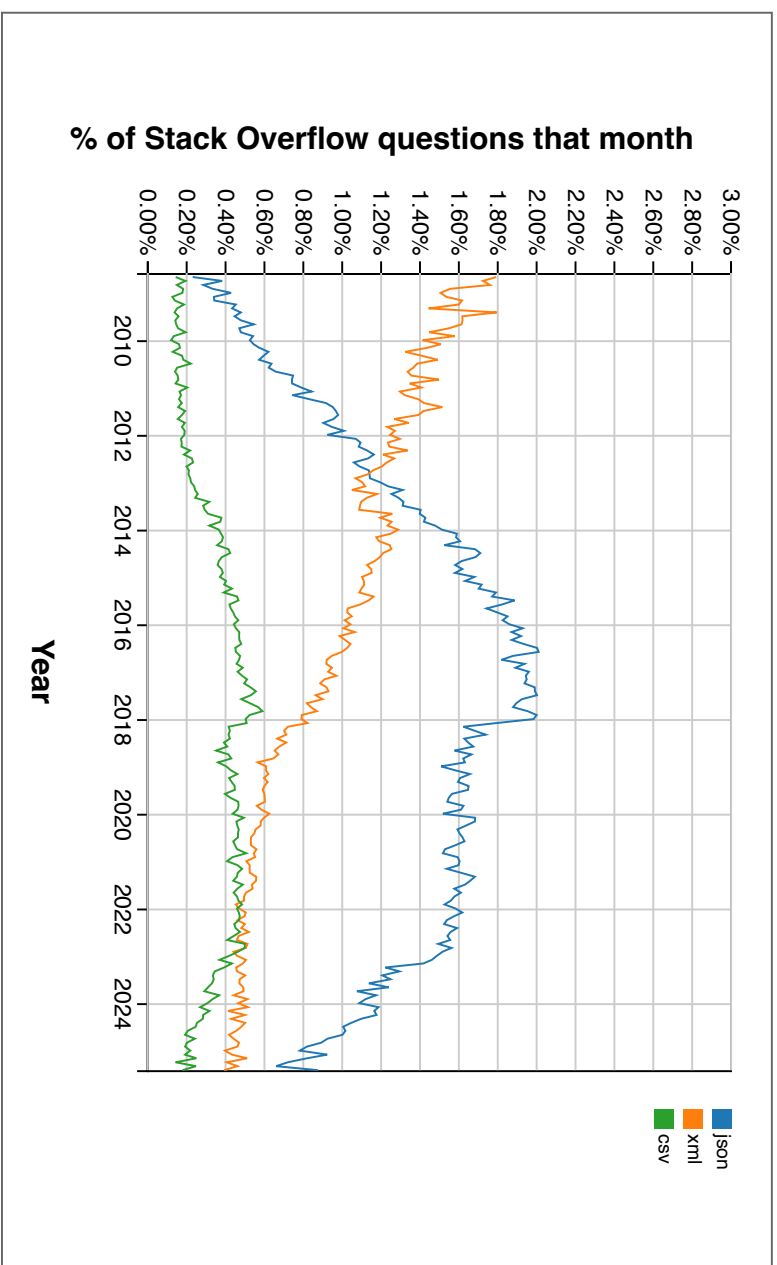
```
<?xml version="1.0"
encoding="UTF-8"?>
<bookstore>
  <book>
    <title lang="en">Harry
Potter</title>
    <price>29.99</price>
  </book>
  <book>
    <title
lang="en">Learning
XML</title>
    <price>39.95</price>
  </book>
</bookstore>
```

Relation	Beispiel	Erklärung
descendant / ancestor	/bookstore/descendant::price	Rekursiv Kindkno- des Element:
child / parent	//book[1]/child::price	Alle Kindkno- des Typs
following / preceding	//book[1]/following::price	Alle Nachfolg des Typs
following- sibling /	//book[1]/following- sibling::book	Alle Nachfolg des Typs

preceding-
sibling

gleichen
Elternkn

WEITERE DATENAUSTAUSCHFORMATE



XML JSON YAML

XML

```
<?xml version="1.0"?>
<person id="123">
  <name>Max</name>
  <age>21</age>
  <city>Karlsruhe</city>
  <hobby>Coding</hobby>
  <hobby>Pizza</hobby>
  <isStudent>true</isStudent>
</person>
```

JSON

```
{
  "id": 123,
  "name": "Max",
  "age": 21,
  "city": "Karlsruhe",
  "hobbies": ["Coding",
              "Pizza"],
  "isStudent": true
}
```

YAML

```
id: 123
name: Max
age: 21
city: Karlsruhe
hobbies:
  - Coding
  - Pizza
isStudent: true
```

JSON XML

	XML	JSON
Struktur	Elemente, Attribute	Key-Value Paare
Lesbarkeit/Effizienz	Mehr "verbose"	Relativ einfach & kompakt
Datentypen	keine (alles Text)	Simple Datentypen (string, number, array, bool)
Metadaten	als Attribute neben dem "eigentlichen" Inhalt	als Key-Value neben Daten
Validierung	DTD, XSD	JSON Schema
Kommentare	Ja	Nein
Einsatz	Ideal für komplexere Dokumente	Simpler, "ad-hoc"- Datenaustausch